

Архитектура для Multi-tenancy в AI SaaS

AI-Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент и планирование проекта

Проектирование и документирование

Качество, интеграции
безопасность

**Продвинутые
архитектурные паттерны**

Инфраструктура

Стратегия, лидерство
и экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Блок 1: Паттерны изоляции

Блок 2: Кастомизация и политики доступа

Блок 3: Практика

Итоги

Q&A

Цели вебинара

К концу занятия вы сможете

1. Понимать, что такое tenant и multi-tenancy

2. Выбирать оптимальный паттерн изоляции (Silo/Pool/Bridge) под конкретный контекст B2B/B2C SaaS

3. Проектировать кастомные политики доступа (RBAC/ABAC) для мультитенантной среды



Смысл

Это нужно уметь чтобы:

1. Понимать, что такое tenant и multi-tenancy, чтобы грамотно спроектировать решение на старте

2. Выбирать оптимальный паттерн изоляции во избежание дорогостоящего рефакторинга через 6-12 месяцев проекта

3. Проектировать кастомные политики доступа для возможности получения клиентов разного уровня (enterprise, b2c и т.д.)

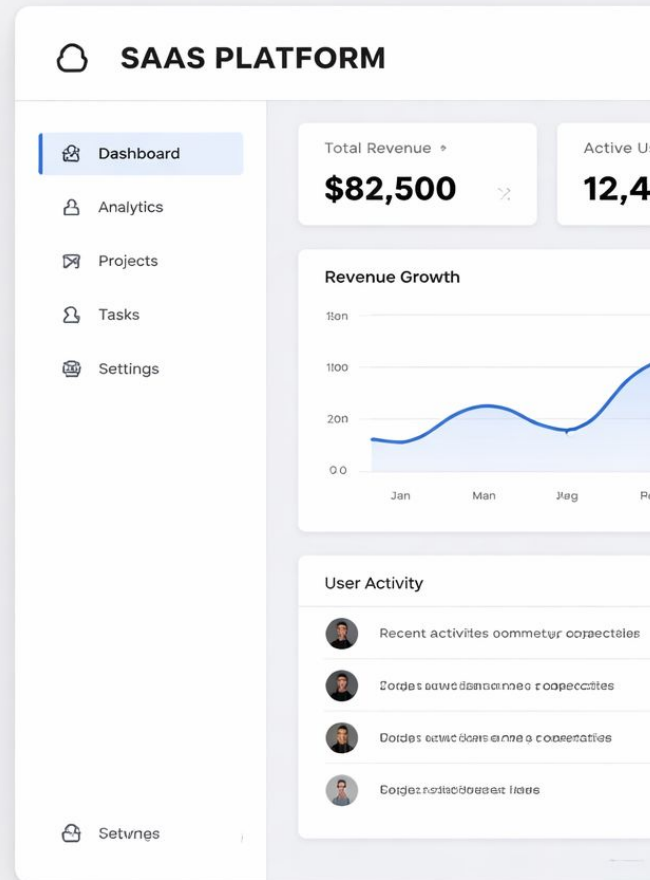


Что такое SaaS?



Software as a Service

Модель предоставления ПО, по которой клиенты получают доступ к готовому приложению через интернет, без необходимости их установки на свои устройства



Multi-tenancy — архитектурный подход, где некоторые компоненты переиспользуются несколькими тенантами (клиентами)



Tenant – это

логический объект, включающий в себя совокупность вычислительных ресурсов, репозиторий и пользователей, выделенных в пользование

Multi-tenancy – гибкий подход



Каждый компонент не является общим. Модель может быть смешанной.



Выбор модели зависит от B2B/B2C, требований к изоляции данных/инфраструктуры и экономике.



Tenant не обязательно является организацией, это может быть также и пространство/проект внутри организации.

Паттерны изоляции

Проблема разделения ресурсов

SaaS-платформа

Приходит крупный
пользователь с желанием
хранить свои данные
отдельно от остальных

Что будем разделять?



Разделение ресурсов

Разделение данных (БД, схемы, таблицы)

Разделение инфраструктуры (compute, сети, окружения)

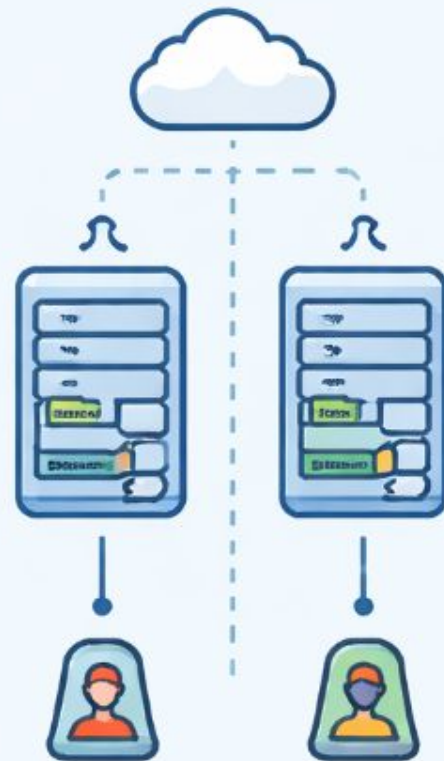
Разделение AI-компонентов (векторные индексы, LLM-ключи, кэши)

Паттерн SILO

🏠 **SILO** — Полная изоляция.

Каждый арендатор получает собственные экземпляры всех ресурсов:

- Отдельная БД (или отдельный кластер)
- Отдельное окружение / namespace / деплой



multitenancy

SILO

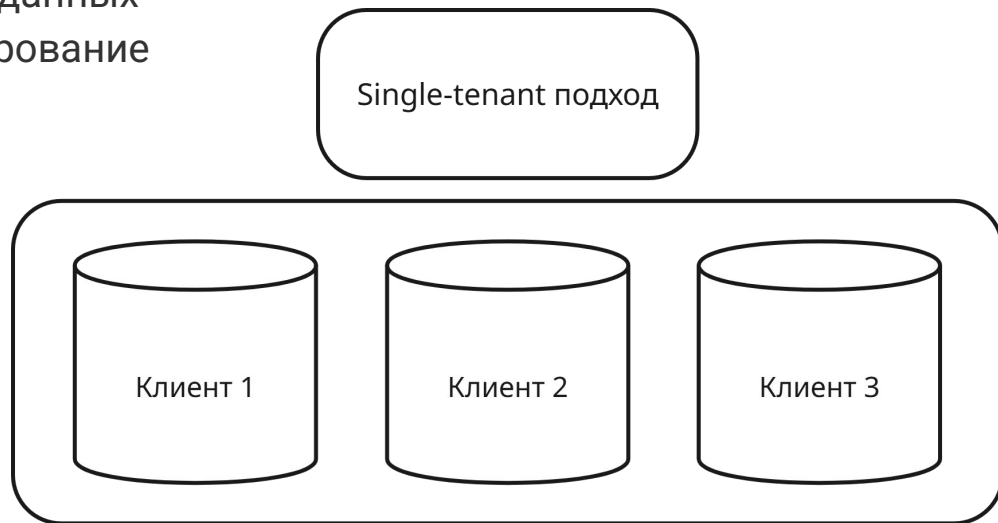
Отдельные Базы Данных

Преимущества:

- Индивидуальность
- Простое резервное копирование данных
- Относительно простое масштабирование

Недостатки:

- Дорого, требует много ресурсов



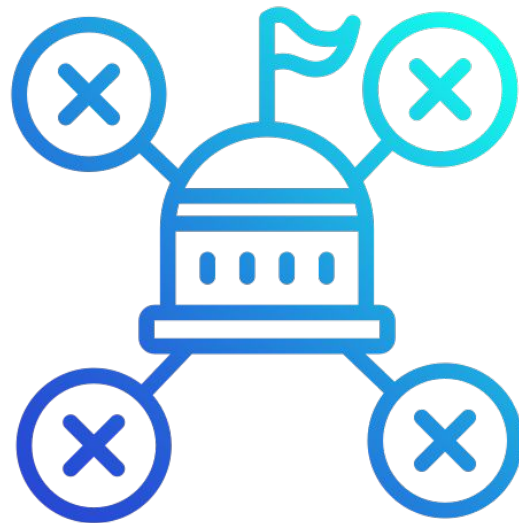
SILO – преимущества

Максимальная изоляция данных и ресурсов

Простое соответствие compliance требованиям
(SOC2, GDPR, HIPAA)

Независимое масштабирование и резервное
копирование

Нет проблемы "noisy neighbour"

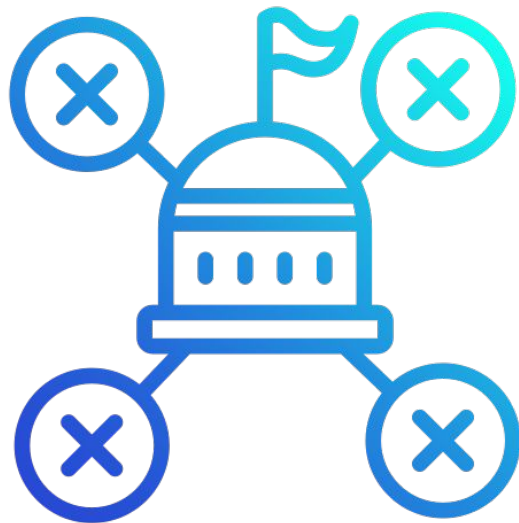


SILO — недостатки

Высокая стоимость (N клиентов = N инфраструктур)

Сложность патчинга и обновлений (каждый инстанс отдельно)

Долгий onboarding нового клиента



Когда выбирать SILO?



Enterprise B2B, healthcare/fintech с жёсткими compliance-требованиями



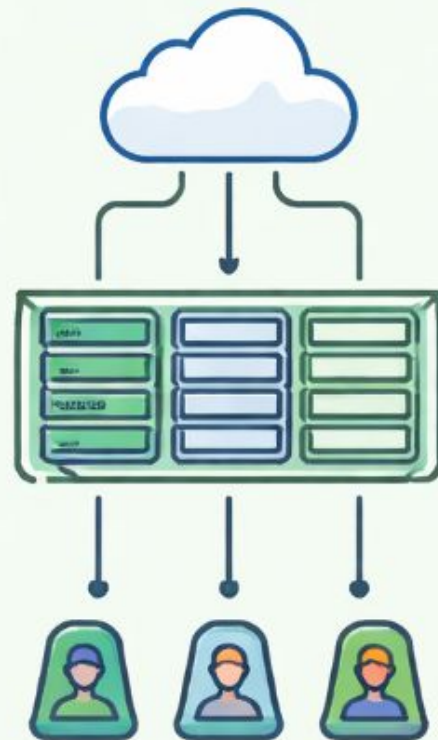
Крупные клиенты с кастомными SLA

Паттерн POOL

🏊‍♂️ **POOL** — Общая инфраструктура.

Все tenants разделяют один экземпляр инфраструктуры. Изоляция на уровне логики приложения.

- Общая БД (возможно разбиение на уровне схем)
- Общее окружение



multitenancy

POOL

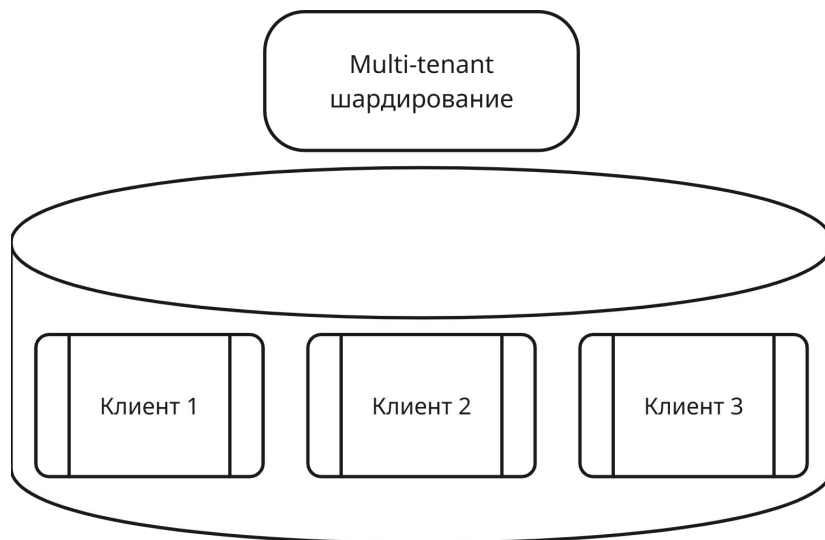
Одна БД – Разные схемы

Преимущества:

- Простая расширяемость и масштабируемость
- Экономия вычислительных ресурсов
- Разделение доступа на уровне СУБД
- Средняя индивидуальность

Недостатки:

- Разделение данных логическое
- Сложности с резервными копиями



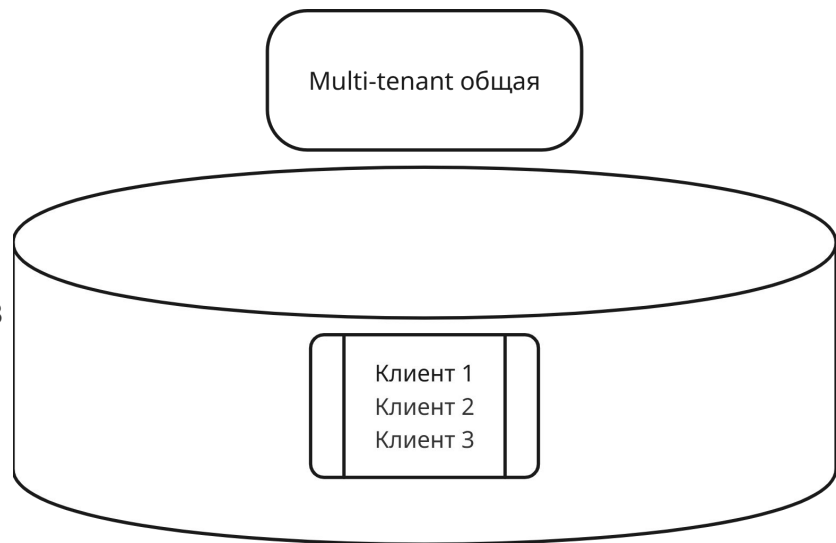
Одна БД – Общая схема

Преимущества:

- Дешево, быстро реализуемо
- Добавление клиента - создание записи в таблице

Недостатки:

- Требуется кастомной логики безопасности
- Нет гибкости
- Резервные копии только для всех клиентов



POOL — преимущества

Минимальные затраты на инфраструктуру

Быстрый onboarding

Централизованные обновления и патчи



POOL — недостатки

Требует тщательной логики безопасности
(Row-Level Security)

Риск "noisy neighbour" — один клиент нагружает
всех

Сложнее соответствовать compliance

Резервное копирование только всей БД целиком



Когда выбирать POOL?



В2С или SMB SaaS



Стартапы



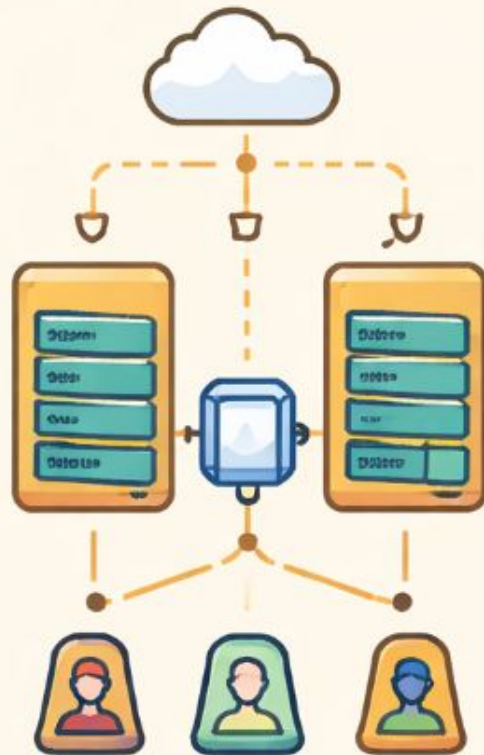
Низкий уровень требований к изоляции

Паттерн BRIDGE

 **BRIDGE** — Гибридный подход.

Часть ресурсов общая, часть изолирована.

Типичная конфигурация: общий application layer + общая основная БД, но отдельные хранилища для чувствительных данных ключевых клиентов.

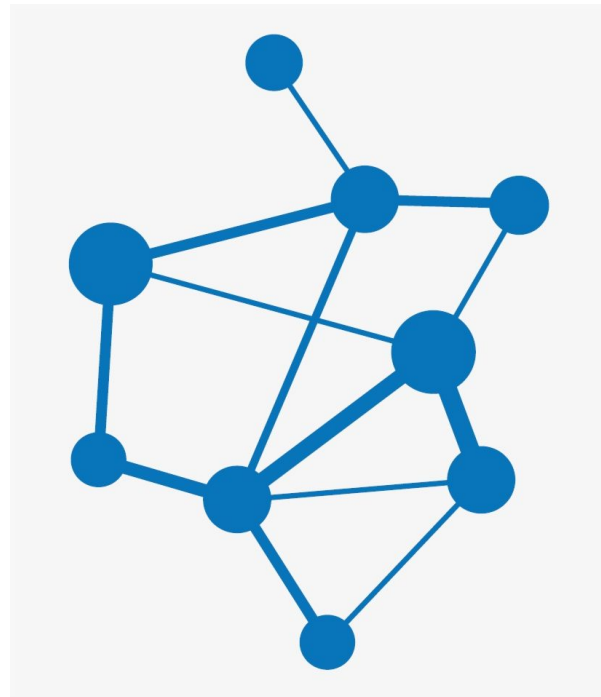


multitenancy
BRIDGE

BRIDGE — преимущества

Гибкость: можно дифференцировать тарифы (Free / Business / Enterprise)

Экономия на большинстве tenants при изоляции VIP-клиентов

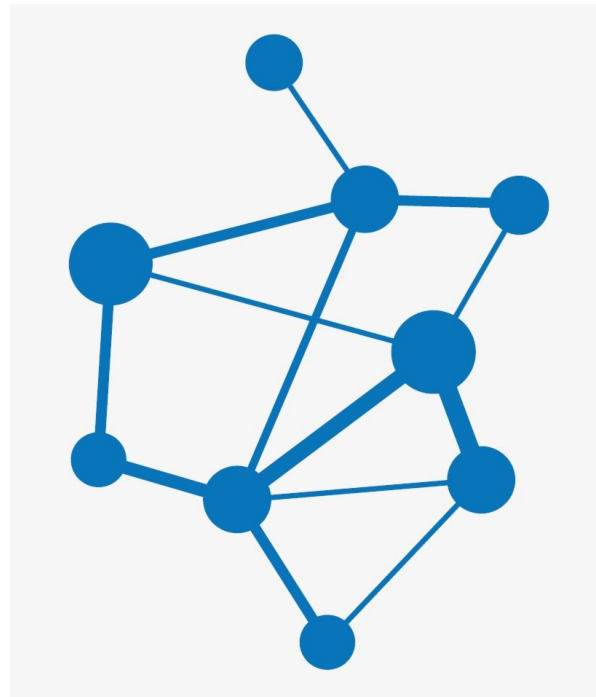


BRIDGE — недостатки

Самая сложная архитектура

Разная логика деплоя для разных тенантов

Сложность мониторинга и дебаггинга



Когда выбирать BRIDGE?



Зрелый SaaS с разными тарифными планами, когда часть клиентов enterprise с требованиями изоляции

Сравнение подходов

Критерий	SILO	POOL	BRIDGE
<i>Изоляция данных</i>	Полная	Логическая	Частичная
<i>Стоимость OPEX</i>	Высокая	Низкая	Средняя
<i>Compliance</i>	Легко	Сложно	Средне
<i>Onboarding</i>	Медленный	Мгновенный	Средний
<i>Масштабирование</i>	На уровне тенанта	Общее	Гибкое
<i>Noisy Neighbour</i>	Нет	Риск	Частично
<i>Сложность</i>	В инфраструктуре	В логике кода	Все вместе

Что добавляет AI-SaaS?



Что добавляет AI-SaaS

AI увеличивает поверхность multi-tenancy



Prompt/response логи

Необходимо отдельно хранить логи для вопросов-ответов, трейсов работы агентов (например, в разные хранилища langfuse)



Embeddings и векторные индексы

Кроме классических СУБД необходимо задуматься о разграничении доступа к векторным данным



Кэш, очереди фоновых задач

Ответы LLM зачастую хранятся в кэше для экономии ресурсов и ускорения времени ответов



Использование LLM-ресурсов

Важно учесть правильную ротацию LLM-ключей и отслеживать статистику использования моделей каждым тенантом

Вопросы



если есть вопросы



если вопросов нет

Кастомизация и политики доступа

Tenant Context Propagation

Источники контекста:

- JWT-токен (`claims.tenant_id`)
- Поддомен (`tenant.myapp.com`)
- Header (`X-Tenant-ID`)

Принципы:

- Tenant context получается **на входе** (API Gateway / middleware)
- Передаётся через весь стек: HTTP headers → сервисы → очереди → векторные запросы
- **Никогда** не берётся из тела запроса от клиента (security risk)

```
@app.middleware("http")
async def tenant_context(request: Request, call_next):
    tenant_id = extract_tenant_from_jwt(request.headers["Authorization"])
    request.state.tenant_id = tenant_id
    response = await call_next(request)
    return response
```

RBAC vs ABAC в multi-tenant контексте

RBAC (Role-Based Access Control). Роли привязаны к тенанту:

tenant_A:admin

tenant_B:viewer

- ✓ Простота реализации
- ✓ Понятно пользователям
- ✗ Плохо масштабируется при сложных правилах

RBAC vs ABAC в multi-tenant контексте

ABAC (Attribute-Based Access Control). Политики на основе атрибутов объекта, субъекта и окружения

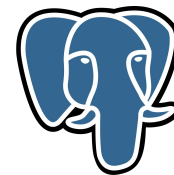
✓ Гибкость: "если документ помечен как confidential и пользователь — manager и в рабочее время"

✓ Мощно для enterprise-кастомизации

✗ Сложность реализации и отладки

Рекомендация: RBAC как основа + ABAC для специфических enterprise-политик

Row-Level Security и Namespace Isolation



Изоляция на уровне данных (PostgreSQL Row-Level Security, Pool модель)

```
1  -- Создание политики
2  CREATE POLICY tenant_isolation ON documents
3      USING (tenant_id = current_setting('app.tenant_id')::uuid);
4
5  -- Применение
6  ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
7
8  -- В коде: устанавливаем контекст перед запросом
9  SET app.tenant_id = '550e8400-e29b-41d4-a716-446655440000';
```


Row-Level Security и Namespace Isolation



Векторные БД (Qdrant)

- **DB per tenant** — Silo-подход, максимальная изоляция
- **Payload filter** — Pool-подход, фильтр по tenant_id при каждом поиске
- **Named vectors** — для Bridge: общие и изолированные векторы

⚠ *Риск: забыть добавить tenant_id фильтр в поиск → кросс-тенантная утечка данных*



Управление LLM-ресурсами по тенантам

Проблемы без управления

- Один тенант "съедает" весь лимит API
- Невозможно понять стоимость по клиентам
- Утечка ключей — удар по всем тенантам

Архитектурные решения

- Пул ключей с ротацией: несколько API-ключей, распределение через LLM-прокси (LiteLLM)
- Token budget per tenant: мягкие и лимиты в Redis (tenant:{id}:tokens_used) или жесткие в реляционной БД
- Cost attribution: логирование токенов в разрезе тенанта для биллинга

Управление LLM-ресурсами по тенантам

Пример реализации “мягких” ограничений

```
async def call_llm(prompt: str, tenant_id: str):  
    # Проверяем бюджет  
    budget = await redis.get(f"tenant:{tenant_id}:budget")  
    if budget <= 0:  
        raise TenantQuotaExceededException()  
  
    response = await llm_client.complete(prompt)  
  
    # Списываем токены  
    await redis.decrby(f"tenant:{tenant_id}:budget", response.usage.total_tokens)  
    return response
```

Observability в Multi-tenant AI SaaS: Что мониторим?



Бизнес-метрики:

- Запросы в сек / арендатор
- Стоимость токенов / арендатор / день
- Latency p95 / арендатор



Трейнсинг:

- tenant_id как обязательный тег в каждом span
- Langfuse: отдельные projects per арендатор или tenant_id в metadata
- Alerting при аномальном потреблении



Security-метрики

- Попытки доступа к данным другого арендатора
- Rate limit нарушения по арендаторам
- Аномалии в prompt-паттернах (prompt injection)

Вопросы



если есть вопросы



если вопросов нет

Практика.

Multi-tenant RAG-сервис

Задание

Контекст: B2B SaaS — корпоративный поиск по документам. Клиенты — компании 50–5000 человек. Три тарифа: Starter, Business, Enterprise.

1. Выберите паттерн изоляции (Silo / Pool / Bridge) — обоснуйте одним предложением
2. Нарисуйте схему (Data Flow): какие компоненты изолированы, какие общие (БД, Vector DB, кэш, логи)
3. Объясните, какими способами вы будете обеспечивать безопасность данных/предотвращать утечки



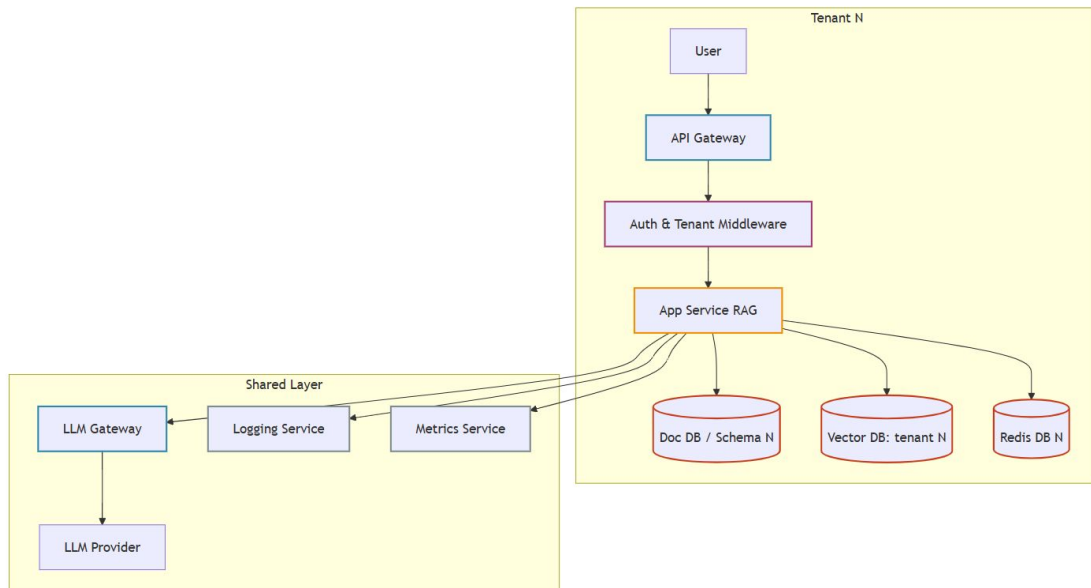
20 мин

время на выполнение



Вариант решения с помощью Bridge

Bridge-паттерн даёт каждому тенанту отдельную «зону» данных (свой кластер/коллекция/схема), при этом общие сервисы (LLM-gateway, логи, метрики) остаются едиными, что удобно для тарифов Starter/Business/Enterprise и постепенного перехода к более серьезной изоляции для Enterprise.



**Подведём
итоги занятия**

Ключевые тезисы занятия

1. Silo/Pool/Bridge — выбор зависит от контекста: B2B/B2C, compliance требования, тарифные планы. “Лучшего” варианта в вакууме нет.
2. AI расширяет поверхность multi-tenancy: векторные индексы, LLM-ключи, кэши и логи — всё требует tenant-aware изоляции
3. Tenant context должен получаться один раз на входе и верифицироваться, а не браться из пользовательского ввода
4. RLS + payload filters — основные механизмы Pool-изоляции; одна пропущенная проверка = утечка
5. Observability с tenant_id — обязательное условие и для security, и для экономики продукта



**Оцените каждое утверждение от 1 до 5,
где 1 = плохо, 5 = очень хорошо**

- A.** Я могу выбрать паттерн изоляции под конкретный SaaS-кейс
- Б.** Я понимаю, как проектировать tenant context propagation
- В.** Я могу спроектировать схему данных для multi-tenant RAG



Отправьте в чат эмодзи, который отражает ваше настроение после занятия



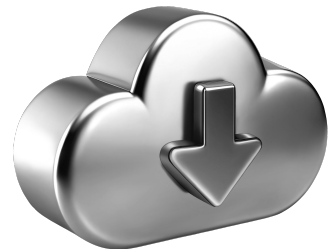
Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Полезные материалы:

1. <https://habr.com/ru/companies/microsoft/articles/145027/>
2. https://docs.litellm.ai/docs/proxy/multi_tenant_architecture
3. https://www.reddit.com/r/SQL/comments/1merufb/postgresql_rowlevel_security_a_beginnerfriendly/?tl=ru
4. O'reilly – [Building Multi-Tenant SaaS Architectures](#)
5. [Статья Microsoft о multitenant AI](#)



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

